

骰子模擬器 Reflecion

A1143344 張鈞甯

1. AI 哪裡有問題？

AI 使用 `(int)(Math.random() * 6) + 1` 來生成隨機數，雖然功能正確，但是在閱覽程式時，會顯得比較複雜不夠清晰。

- 改了哪裡？

將`(int)(Math.random() * 6) + 1` 改成建立 `Random` 物件，並使用：名稱`.nextInt(6) + 1`，來生成 1~6 的隨機數。

- 為什麼改？

因為這樣的方式會讓整個程式的邏輯比較清晰，也不用再強制型別轉換，在閱覽程式時，能很清楚地便了解這行程式是在做什麼。

2. AI 哪裡有問題？

使用 `GridLayout` 來排版，會讓按鈕的大小直接占了視窗的 1/3，較不美觀。



- 改了什麼？

原本在容器裡使用 `GridLayout` 直接將兩個標籤及一個按鈕由上而下分成三等份來排列，改成再多建立一個容器 `btnPanel`，裡面放擲骰子的按鈕，接著將這個容器 `btnPanel` 放進原本的容器中。

原本：

```
static JPanel pne=new JPanel(new GridLayout(3, 1));  
pne.add(lab2);    //添加最上方的標籤至容器  
pne.add(lab1);    //添加中間的標籤至容器  
pne.add(btn);     //添加按鈕至容器
```

改成：

```
static JPanel pne=new JPanel(new GridLayout(3, 1));
```

```
static JPanel btnPanel = new JPanel();  
pne.add(lab2);      //添加最上方的標籤至容器 pne  
pne.add(lab1);      //添加中間的標籤至容器 pne  
btnPanel.add(btn);   //添加按鈕至容器 btnPanel  
pne.add(btnPanel);   //添加容器 btnPanel 至容器 pne
```

- 為什麼改？

這樣既能讓視窗中的兩個標籤及一個按鈕由上而下均勻分布至視窗中，還能讓按鈕的尺寸不那麼大，也不會緊貼視窗底部，畫面看起來較協調、美觀。

